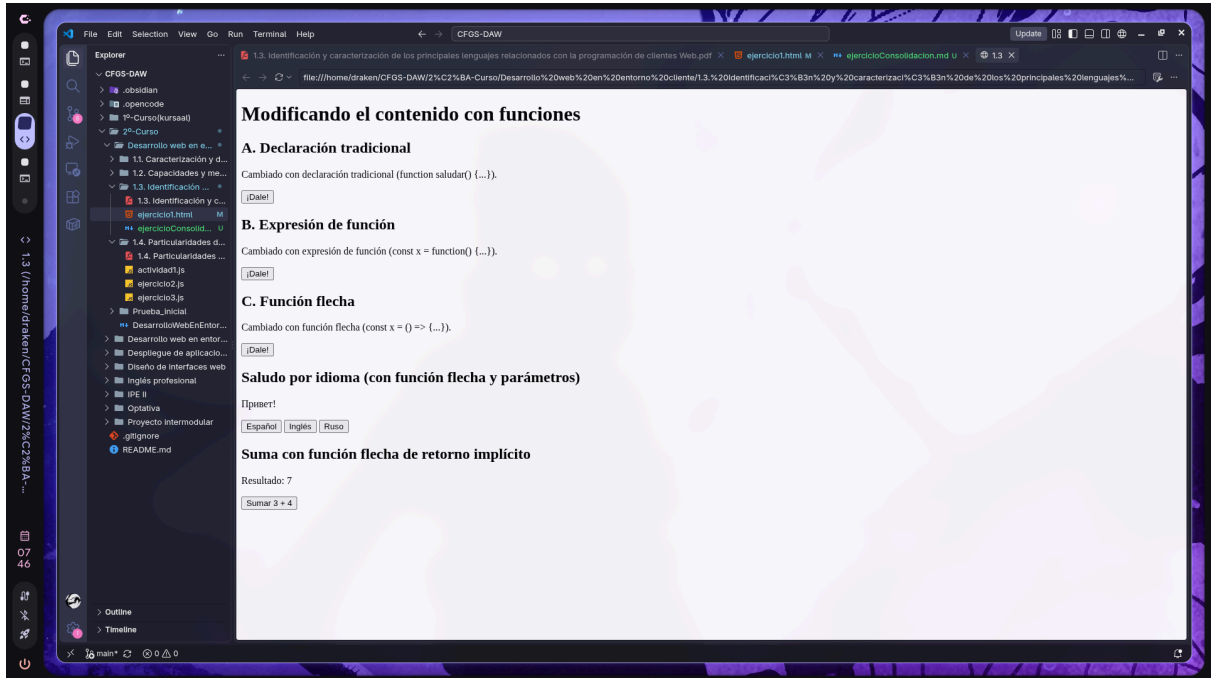
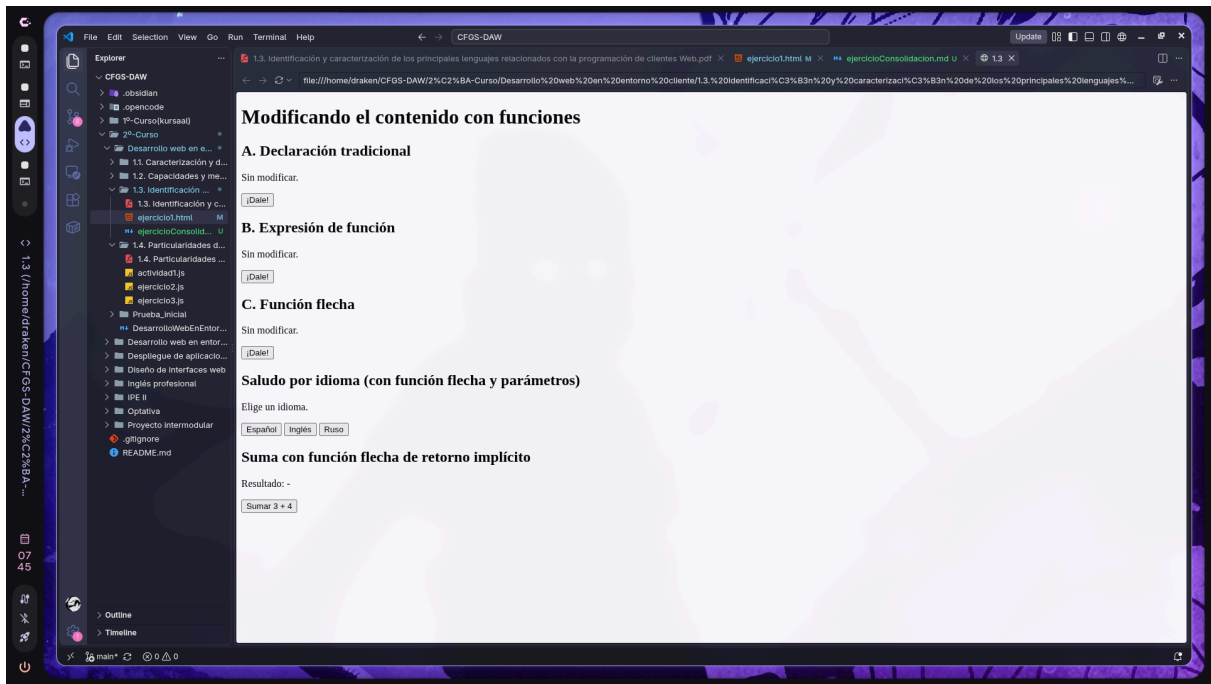


```
1 <!doctype html>
2 <html lang="es">
3 <head>
4 <meta charset="UTF-8" />
5 <title>1.3</title>
6 </head>
7 <body>
8 <h1>Modificando el contenido con funciones</h1>
9
10 <!-- A. Declaración tradicional (function declaration): disfruta de hoisting -->
11 <h2>A. Declaración tradicional</h2>
12 <p id="tradicional">Sin modificar.</p>
13 <button type="button" onclick="cambiarTextoTradicional()">¡ Dale! </button>
14
15 <!-- B. Expresión de función: asignada a una constante, no se puede usar antes de definirla -->
16 <h2>B. Expresión de función</h2>
17 <p id="expresion">Sin modificar.</p>
18 <button type="button" onclick="cambiarTextoExpresion()">¡ Dale! </button>
19
20 <!-- C. Función flecha (ES6): sintaxis compacta -->
21 <h2>C. Función flecha</h2>
22 <p id="flecha">Sin modificar.</p>
23 <button type="button" onclick="cambiarTextoFlecha()">¡ Dale! </button>
24
25 <h2>Saludo por idioma (con función flecha y parámetros)</h2>
26 <p id="saludo">Elige un idioma.</p>
27 <button type="button" onclick="saludar('es')">Español</button>
28 <button type="button" onclick="saludar('en')">Inglés</button>
29 <button type="button" onclick="saludar('ru')">Ruso</button>
30
31 <h2>Suma con función flecha de retorno implícito</h2>
32 <p id="suma">Resultado: </p>
33 <button type="button" onclick="mostrarSuma(3, 4)">Sumar 3 + 4</button>
34
35 <script>
36 // A. Function Declaration
37 function cambiarTextoTradicional() {
38   document.getElementById("tradicional").innerHTML =
39     "Cambiado con declaración tradicional (function saludar() {...}).";
40 }
41
42 // B. Function Expression
43 const cambiarTextoExpresion = function () {
44   document.getElementById("expresion").innerHTML =
45     "Cambiado con expresión de función (const x = function() {...}).";
46 };
47
```

```
1 <html lang="es">
2 <body>
3
4 <h2>Saludo por idioma (con función flecha y parámetros)</h2>
5 <p id="saludo">Elige un idioma.</p>
6 <button type="button" onclick="saludar('es')">Español</button>
7 <button type="button" onclick="saludar('en')">Inglés</button>
8 <button type="button" onclick="saludar('ru')">Ruso</button>
9
10 <h2>Suma con función flecha de retorno implícito</h2>
11 <p id="suma">Resultado: </p>
12 <button type="button" onclick="mostrarSuma(3, 4)">Sumar 3 + 4</button>
13
14 <script>
15 // A. Function Declaration
16 function cambiarTextoTradicional() {
17   document.getElementById("tradicional").innerHTML =
18     "Cambiado con declaración tradicional (function saludar() {...}).";
19 }
20
21 // B. Function Expression
22 const cambiarTextoExpresion = function () {
23   document.getElementById("expresion").innerHTML =
24     "Cambiado con expresión de función (const x = function() {...}).";
25 };
26
27 // C. Arrow Function
28 const cambiarTextoFlecha = () => {
29   document.getElementById("flecha").innerHTML =
30     "Cambiado con función flecha (const x = () => {...}).";
31 };
32
33 // Función flecha con parámetro
34 const saludos = { es: "¡Hola!", en: "Hello!", ru: "Привет!" };
35 const saludar = (idioma) => {
36   document.getElementById("saludo").innerHTML = saludos[idioma];
37 };
38
39 // Función flecha de retorno implícito (una sola línea, sin return ni llaves)
40 const sumar = (a, b) => a + b;
41 const mostrarSuma = (a, b) => {
42   document.getElementById("suma").innerHTML = "Resultado: " + sumar(a, b);
43 };
44 </script>
45 </body>
46 </html>
47
```



ACTIVIDAD PROPUESTA 1.1: ¿QUÉ ES LA PROGRAMACIÓN REACTIVA?

La programación reactiva es un paradigma en el que se declaran flujos de datos y sus dependencias, y el propio sistema se encarga de propagar automáticamente los cambios por toda esa cadena de dependencias cuando el dato de origen cambia — sin que el programador tenga que actualizar manualmente cada elemento afectado.

Cómo se comporta una hoja de cálculo (ejemplo):

Si en una hoja tenemos $A1 = 5$, $B1 = 10$ y $C1 = A1 + B1$ (resultado 15), y cambiamos $A1$ a 20, $C1$ se recalcula solo a 30 de forma inmediata, sin que el usuario tenga que pulsar nada ni volver a escribir la fórmula. La hoja de cálculo detecta que $C1$ depende de $A1$, y reacciona al cambio propagando el nuevo valor por toda la cadena de celdas que dependan de ella (incluidas las que a su vez dependan de $C1$).

Relación con los frameworks modernos: en React, Angular o Vue ocurre lo mismo con el estado de la aplicación. El estado hace de $A1$: cuando cambia, todos los elementos de la interfaz que "dependen" de él (el DOM, variables derivadas, otros componentes) se actualizan solos — igual que $C1$ se recalcula solo — sin que el programador tenga que escribir manualmente el código que sincroniza cada parte de la pantalla con el dato nuevo.

TABLA DE DECISIÓN

1. Escenario: Pequeña tienda de barrio, presupuesto reducido y despliegue rápido

Elección: Vue.js

Justificación: Es el único de los tres con curva de aprendizaje progresiva, descrito en el tema como mucho más accesible y suave que la de Angular: un equipo pequeño puede ponerse a producir sin dominar inyección de dependencias, TypeScript ni RxJS. Está pensado además priorizando la ligereza y la velocidad de ejecución, y aporta patrones de diseño, componentes prediseñados, gestión de rutas y estructuras comunes ya resueltas, que el tema relaciona con la velocidad de entrega (Time to Market). El coste económico nulo (open-source y gratuito) hace que el presupuesto reducido no penalice la licencia; lo que se evita es la complejidad técnica innecesaria.

2. Escenario: Portal bancario de una entidad financiera, cientos de programadores y tipado robusto estricto

Elección: Angular

Justificación: El tema relaciona los dos datos del enunciado: debido a los riesgos del dinamismo extremo en aplicaciones complejas (especialmente financieras), el estándar actual de la industria es utilizar TypeScript. Angular es el único de los tres programado en TypeScript, superconjunto tipado que permite escribir código seguro con tipos fijos que se comprueban mientras programas, con interfaces y tipado estático. Además, su rigidez estructural aporta estandarización de equipos, ya que permite que nuevos programadores se incorporen a un proyecto en marcha de forma eficiente si ya conocen el estándar del framework, factor decisivo con cientos de programadores.

3. Escenario: Aplicación interactiva con renderizado ultra rápido de miles de productos y cambios constantes

Elección: ReactJS

Justificación: El criterio decisivo es el rendimiento, y aquí ReactJS resuelve directamente el problema que describe el tema: manipular el DOM nativo es una operación lenta porque obliga al motor a recalcular geometrías y repintar píxeles. ReactJS mantiene una copia ligera del DOM en RAM (DOM Virtual) y, al cambiar los datos, calcula las diferencias mínimas entre el DOM virtual y el real (reconciliación), actualizando únicamente los nodos estrictamente necesarios. Con miles de productos y cambios constantes, esa actualización granular evita el repintado masivo. Además, la programación orientada a componentes (una tarjeta por producto, cada una con su propio estado) es el modelo que permite sostener ese volumen de datos.